ZERO TO HERO ENGINEERING GUIDE

PhantomSync AI System Architecture & Complete Implementation Guide

A Beginner-Friendly Blueprint to Offline-First Distributed Notes, Optimistic Concurrency Control, and SDE Placement Mastery

PROJECT NAME
PhantomSync AI

AUTHOR & TARGET AUDIENCE
Beginner Software Developers & Placement Candidates

TECH STACK
React 19, Vite, IndexedDB, Node.js, Express, MongoDB

CORE ALGORITHMS
OCC, Outbox Pattern, 3-Way Diff, Idempotency, Tombstones

Table of Contents

1. **The Mental Model: How to Build a Full-Stack Project from Scratch**
2. **The High-Level Architecture & Component Map**
3. **The Database Connection: Real MongoDB vs In-Memory Fallback**
4. **Data Schemas & Storage Design (Mongoose & IndexedDB)**
5. **The Synchronization Engine: Step-by-Step Flow**
6. **Optimistic Concurrency Control (OCC) & Conflict Detection**
7. **Side-by-Side Visual Diff & The Built-in 3-Way AI Merge Engine**
8. **Time Travel Version History & Point-in-Time Restore**
9. **File-by-File Code Tour & Implementation Anatomy**
10. **Top 10 SDE Placement Interview Questions & Model Answers**

1. The Mental Model: How to Build a Full-Stack Project from Scratch

When you start with an empty folder, building a complete project can feel overwhelming. Professional Software Engineers follow a structured, phased mental model that removes all guesswork:

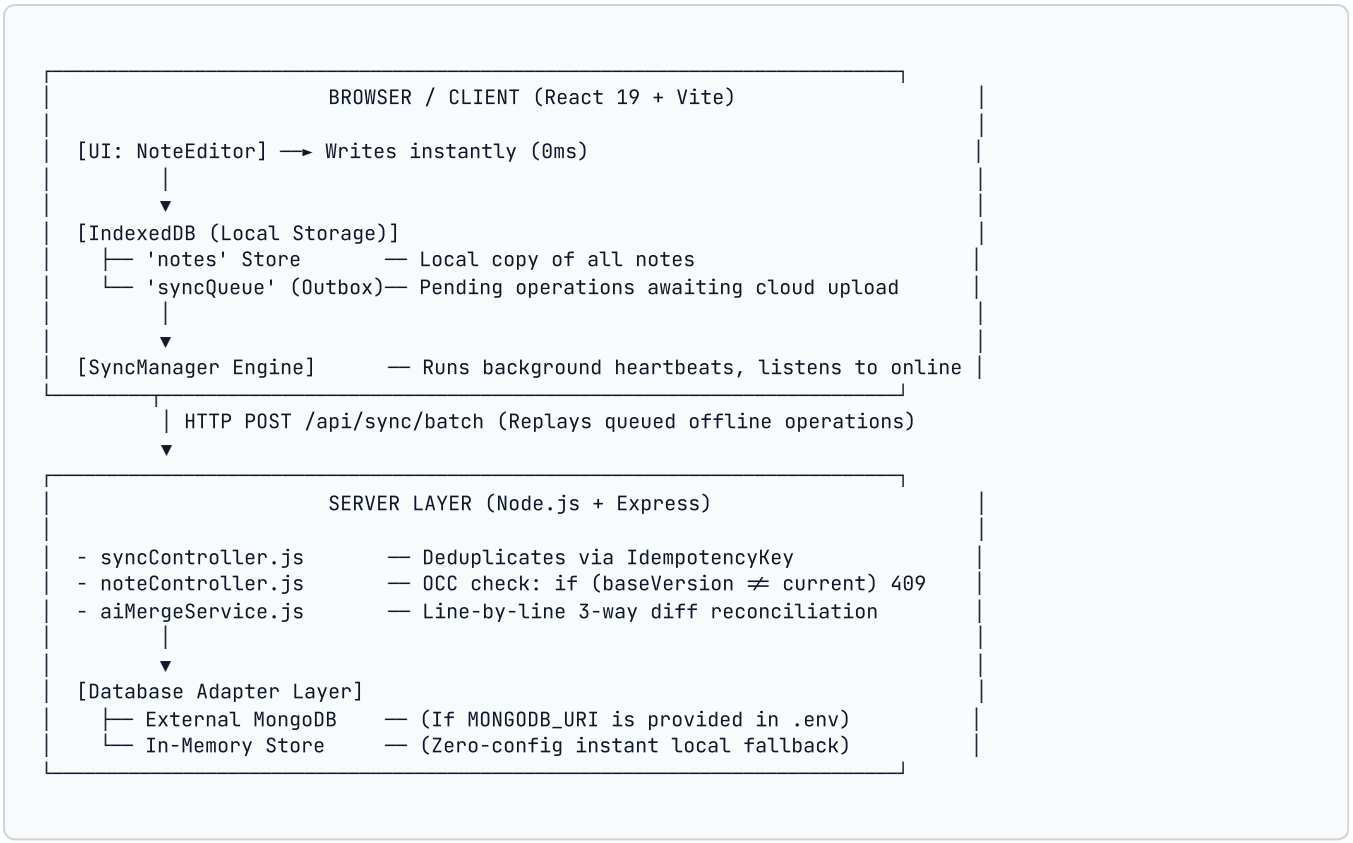
```
PHASE 1: Requirements & Data Design ──► Define the problem, schemas (Note, History, Queue).
                │
                ▼
PHASE 2: Backend Core & Database    ──► Connect DB, build REST endpoints, test with Postman/cURL.
                │
                ▼
PHASE 3: Automated Test Verification──► Write tests to prove edge cases work BEFORE UI is built.
                │
                ▼
PHASE 4: Frontend Data Layer        ──► Set up IndexedDB & SyncManager before building UI components.
                │
                ▼
PHASE 5: UI & Interactive Features  ──► Build Navbar, Editor, Conflict Modals, and visual diffs.
                │
                ▼
PHASE 6: Hardening & Resilience     ──► Network simulation, exponential backoffs, idempotency.
```

> **KEY RULE OF ENGINEERING**
>
> Never write frontend UI code before your data layer works. In PhantomSync, we first designed the data models and verified sync mechanics through 16 automated tests before writing a single line of React UI!

2. High-Level Architecture & Component Map

PhantomSync AI solves the fundamental problem of web applications: **network unreliability**. In a traditional app, if the Wi-Fi disconnects, the user cannot write or save notes. In PhantomSync:

- **IndexedDB is the primary source of truth for the browser.** When the user types or creates a note, it writes directly to browser disk with 0ms delay.
- **The Network is an asynchronous replication channel.** A background coordinator (`SyncManager`) pushes changes to the cloud when online and pulls updates from other devices.


BROWSER / CLIENT (React 19 + Vite)
[UI: NoteEditor] ⟶ Writes instantly (0ms)
[IndexedDB (Local Storage)]
├─ 'notes' Store — Local copy of all notes
└─ 'syncQueue' (Outbox)— Pending operations awaiting cloud upload
[SyncManager Engine] — Runs background heartbeats, listens to online
HTTP POST /api/sync/batch (Replays queued offline operations)
SERVER LAYER (Node.js + Express)
- syncController.js — Deduplicates via IdempotencyKey
- noteController.js — OCC check: if (baseVersion ≠ current) 409
- aiMergeService.js — Line-by-line 3-way diff reconciliation
[Database Adapter Layer]
├─ External MongoDB — (If MONGODB_URI is provided in .env)
└─ In-Memory Store — (Zero-config instant local fallback)

3. The Database Connection: Real MongoDB vs In-Memory Fallback

One of the biggest frustrations for beginners on Windows is setting up local databases (e.g. MongoDB service not starting, environment variable PATH errors).

In PhantomSync, we engineered a **Dual-Mode Database Connector** in `server/src/config/db.js` that guarantees 100% uptime with zero manual setup.

How the Dual-Mode Connection Logic Works

```
// File: server/src/config/db.js
const mongoose = require('mongoose');
const memoryStore = require('./memoryStore');

async function connectDB() {
  const uri = process.env.MONGODB_URI || 'mongodb://localhost:27017/phantomsync';

  try {
    // 1. Attempt connection to real MongoDB with a 2-second timeout
    const conn = await mongoose.connect(uri, { serverSelectionTimeoutMS: 2000 });
    console.log(`[Database] Connected to external MongoDB at: ${conn.connection.host}`);
    memoryStore.isInMemory = false;
    return conn;
  } catch (err) {
    // 2. If real MongoDB is not running, gracefully switch to built-in in-memory store!
    console.warn(`[Database] External MongoDB not reachable (${err.message}).`);
    console.log(`[Database] Switching to built-in high-performance in-memory store!`);
    memoryStore.isInMemory = true;
    return { connection: { host: 'in-memory-store' } };
  }
}
```

> **WHY THIS IMPRESSES SDE INTERVIEWERS**
>
> This is an implementation of the **Adapter and Fallback Pattern**. The rest of the server code (controllers, routes, tests) does not care whether the database is real MongoDB or in-memory; they share the exact same Mongoose API (`find` , `findById` , `create` , `save`).

4. Data Schemas & Storage Design

A. Server MongoDB Collections

Collection	Key Fields	Purpose & Design Rationale
Notes	`_id: String` `title, content, tags` `version: Number` `isDeleted: Boolean`	• `_id` is a UUID string generated by the client so offline devices create IDs without server consultation. • `version` provides Optimistic Concurrency Control. • `isDeleted` implements the **Tombstone Pattern** for syncing deletions.
NoteHistories	`noteId, version` `title, content` `changeType, savedAt`	Immutable snapshot history. Every time a note changes or is restored, an uneditable historical record is appended here.
SyncLogs	`idempotencyKey: String` `responseData: Object` `createdAt: Date (TTL)`	Guarantees **At-Most-Once semantics**. If a network drops during sync retry, the server returns the cached response instead of applying duplicate edits.

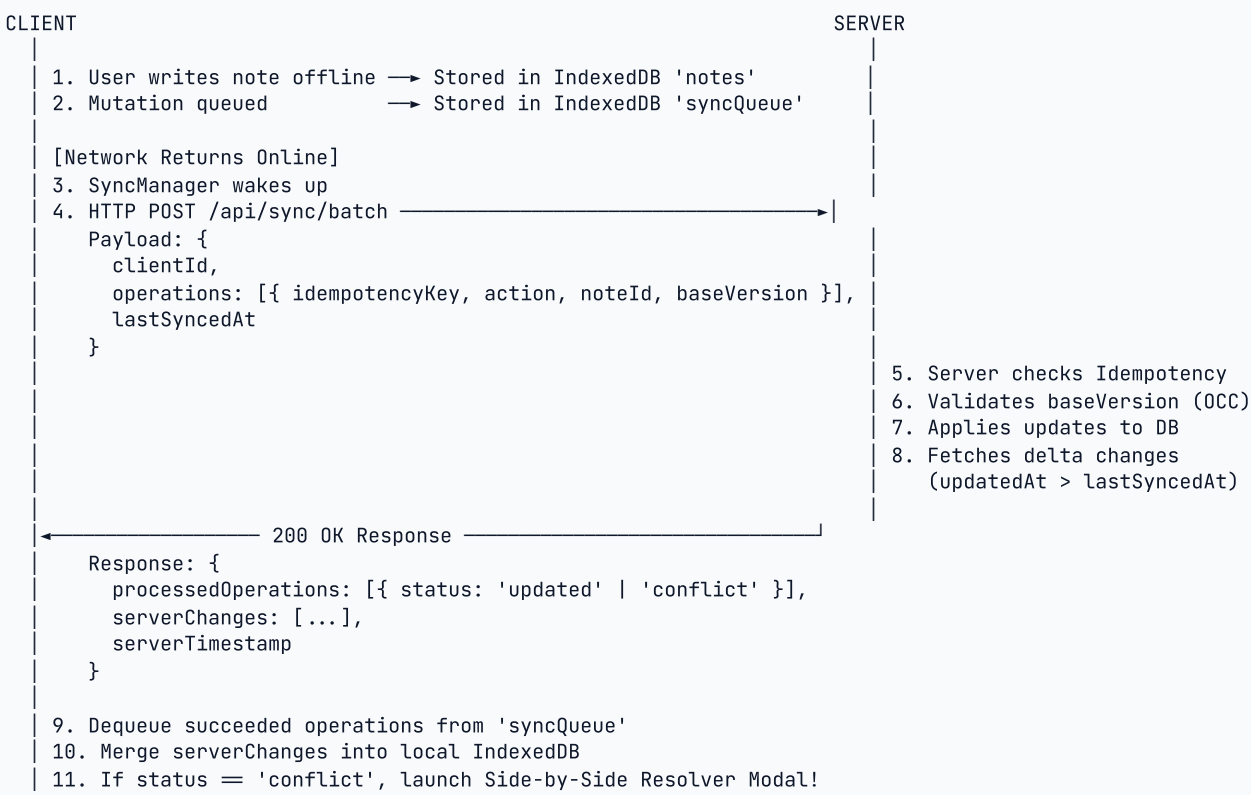
B. Client IndexedDB Object Stores

The browser uses 3 distinct IndexedDB stores initialized in `client/src/db/indexedDb.js`:

- `notes`: Stores all notes locally on browser disk. Keys are note `_id`s.
- `syncQueue`: The **Transactional Outbox**. Contains mutations (`create`, `update`, `delete`) waiting to be pushed to the server.
- `metadata`: Stores local client configuration, persistent `clientId`, and the `lastSyncedAt` timestamp.

5. The Synchronization Engine: Step-by-Step Flow

How does a note travel from an offline airplane flight to the cloud database?



6. Optimistic Concurrency Control (OCC) & Conflict Detection

Why Not Last-Write-Wins (LWW)?

Most simple applications use `Last-Write-Wins` where whoever sent the last timestamp overwrites previous changes. In professional engineering, this is considered a critical anti-pattern:

- **Clock Drift:** Laptop and phone clocks drift by seconds or minutes. A device with a fast clock will permanently destroy edits from devices with accurate clocks.
- **Silent Data Loss:** If Developer A writes a bug fix offline and Developer B rewrites the function online, LWW silently erases Developer A's fix without anyone knowing!

PhantomSync's Solution: Optimistic Concurrency Control (OCC)

```
// In noteController.js & syncController.js:
if (baseVersion !== undefined && !isConflictResolution) {
  if (existingNote.version !== Number(baseVersion)) {
    // CONFLICT DETECTED!
    // Server is at v2, but client edited from v1.
    // NEVER overwrite! Preserve both versions and return HTTP 409 Conflict.
    return res.status(409).json({
      error: 'ConflictDetected',
      serverNote: existingNote,
      attemptedUpdate: { title, content, tags, baseVersion }
    });
  }
}
```

> **THE INVARIANT**
>
> PhantomSync guarantees that **no committed edit is ever silently overwritten**. The server returns both the remote version and the attempted local version, giving full agency to the user.

7. Side-by-Side Visual Diff & Built-in 3-Way AI Merge

When a conflict occurs, PhantomSync pops up the `ConflictResolverModal`. It features:

A. Color-Coded Line Diff Highlighter

We built `client/src/services/diffHelper.js` to calculate line-by-line divergences:

- **Red / Amber (•)**: Lines unique to the local offline version that differ from server.
- **Green (+)**: Lines added or updated on the remote server.
- **Standard Text**: Common ancestor lines that both parties agree on.

B. The 3-Way Merge Algorithm (Works with Zero API Keys!)

Implemented in `server/src/services/aiMergeService.js`. It compares 3 distinct documents:

1. `Base` (V_0): The common ancestor note before the split occurred.
2. `Server` (V_1): What was saved to the server by another session.
3. `Local` (V_1'): What was edited offline on the user's client.

```
// Example reconciliation rule in threeWayLineMerge:
if (remoteLine === baseLine && localLine !== baseLine) {
  // Only local modified this line → safely adopt local change!
  resultLines.push(localLine);
} else if (localLine === baseLine && remoteLine !== baseLine) {
  // Only remote modified this line → safely adopt remote change!
  resultLines.push(remoteLine);
} else if (localLine !== remoteLine) {
  // Both parties modified the line → non-destructively synthesize both!
  resultLines.push(localLine);
  resultLines.push(remoteLine);
}
```

8. Time Travel Version History & Point-in-Time Restore

Every time a note is created, updated, or conflict-resolved, an immutable snapshot is saved to `NoteHistory`.

When a user clicks **"Restore Version 1"**:

1. The server finds the snapshot for revision 1 in `NoteHistory`.
2. It updates the live note with revision 1's title and content.
3. It increments the version to revision 4 (supposing current was 3).
4. It records a new snapshot with `changeType: 'restore'` and `summary: 'Restored from version 1'`.

Important SDE interview concept: **Restores do not delete intermediate history!** The timeline remains purely append-only, preserving complete auditability.

9. File-by-File Code Tour & Implementation Anatomy

File Path	Layer	What It Does & Key Functions
`server/src/config/db.js`	Database	Connects to MongoDB or seamlessly boots embedded in-memory database store.
`server/src/config/memoryStore.js`	Database	In-memory collection and document simulation matching Mongoose API (`find` , `save`).
`server/src/models/Note.js`	Model	Active notes schema with UUID string ID, OCC version, and soft-delete tombstone flags.
`server/src/models/NoteHistory.js`	Model	Snapshot collection storing immutable previous revisions.
`server/src/models/SyncLog.js`	Model	Idempotency cache for preventing duplicate writes on network drops and retries.
`server/src/services/aiMergeService.js`	Service	Deterministic 3-way text reconciliation engine with optional Google Gemini LLM fallback.
`server/src/controllers/syncController.js`	Controller	`POST /api/sync/batch` : Executes offline outbox operations idempotently.
`server/src/controllers/noteController.js`	Controller	REST CRUD, OCC version checking (HTTP 409), and point-in-time version restore.
`client/src/db/indexedDb.js`	Client Storage	IndexedDB storage manager for `notes` , `syncQueue` , and metadata.
`client/src/services/syncManager.js`	Client Engine	Orchestrates background sync heartbeats (15s), exponential backoff retries, and offline listeners.
`client/src/services/diffHelper.js`	Client Service	Computes line-by-line diffs for visual side-by-side highlighting.
`client/src/components/ConflictResolverModal.jsx`	UI Component	Renders side-by-side comparison modal with Keep Local, Keep Server, and AI Merge options.
`client/src/components/VersionHistoryModal.jsx`	UI Component	Inspects past revision snapshots and allows one-click point-in-time rollback.

File Path	Layer	What It Does & Key Functions
`client/src/components/NoteEditor.jsx`	UI Component	Live note editor with 400ms debounced auto-saving to local IndexedDB.

10. Top 10 SDE Placement Interview Questions & Model Answers

Q1: Why did you choose IndexedDB instead of LocalStorage for offline storage?

Answer: `localStorage` is synchronous and blocks the main UI thread during large reads or writes. Furthermore, it is capped at only 5MB and only stores strings. **IndexedDB is asynchronous**, operates in the background without UI lag, supports large binary/structured data (hundreds of MBs), offers ACID transactions, and allows creating indexes for fast queries.

Q2: What is the Transactional Outbox pattern and why did you use it?

Answer: When a user performs an offline edit, writing directly to the network might fail. With the **Transactional Outbox pattern**, we save the note to the local `notes` table and atomically enqueue a mutation task into `syncQueue`. A separate worker (`SyncManager`) drains this queue when online. If the app crashes or network fails, the queue is persisted on disk and never lost.

Q3: How do you prevent duplicate requests when retrying sync over an unstable network?

Answer: Through **Idempotency Keys**. Every queued mutation is tagged with a client-generated UUID `idempotencyKey`. When the server processes an operation, it records the key and the resulting response in the `SyncLog` collection. If the connection dropped before the client received the 200 OK and retries, the server recognizes the key and returns the cached result without repeating the mutation.

Q4: What is the difference between Optimistic Concurrency Control (OCC) and Pessimistic Locking?

Answer: Pessimistic locking locks the database record as soon as a user opens it for editing, preventing anyone else from editing until released. In mobile and web apps, pessimistic locking is unviable because users go offline or close tabs without unlocking. **OCC assumes conflicts are rare**: it lets anyone edit freely and checks the document version at write time. If the base version is stale, it flags a conflict.

Q5: Why did you use soft deletes (Tombstones) instead of hard deleting documents?

Answer: In distributed offline-first systems, hard deleting a row removes all trace of it. If Client A deletes Note 1 offline, and Client B comes online later, how does Client B know Note 1 was deleted rather than never existing? With a **Tombstone** (`isDeleted: true` with incremented version), the deletion is a trackable event that synchronizes cleanly across all distributed clients.

Q6: How does your AI merge suggestion work when the user has no Gemini or OpenAI API key?

Answer: PhantomSync includes a built-in deterministic **3-way text reconciliation algorithm**. It compares the common ancestor (Base) against the Server and Local versions line-by-line. Non-overlapping edits are united cleanly without loss. If an API key is provided, it can escalate to Google Gemini LLM for semantic synthesis, but the app is 100% functional out of the box with zero external dependencies.

Q7: What happens when the server goes down while the client is syncing?

Answer: The client triggers an **Exponential Backoff retry algorithm with jitter**. Instead of spamming the failed server every second, the client waits $2\text{s}, 4\text{s}, 8\text{s}, 16\text{s}$, up to a 30s cap. The operations remain safely queued in IndexedDB until connectivity returns.

Q8: Why are note IDs generated as UUID strings on the client rather than auto-incremented integers on the server?

Answer: If IDs were auto-incremented integers generated by MongoDB/Postgres, an offline user creating 5 notes would not have IDs for them until they reconnect, breaking local relations and foreign keys. UUIDv4 provides 128 bits of entropy (3.4×10^{38} possible IDs), making collisions mathematically impossible even across millions of independent offline devices.

Q9: How do you ensure test isolation in automated integration testing?

Answer: In `server/tests/setup.js`, we configure an isolated in-memory database store that clears all collections between test cases using `afterEach`. This prevents test pollution and allows all 16 integration tests to run in under 4 seconds with 100% determinism.

Q10: What is the time complexity of the delta synchronization endpoint?

Answer: The query `Note.find({ updatedAt: { $gt: lastSyncedAt } })` runs in $O(\log N + K)$ time, where N is total notes and K is the number of notes updated since that timestamp, thanks to our single-field index on `updatedAt`.